

MedLedger: Secure Medical Data Integrity and Access Control

Using Distributed Ledger Architecture

Enhanced Technical Specification and Implementation Guide

1. Executive Summary & Technical Overview

MedLedger is a hybrid blockchain-based healthcare data management system that combines distributed ledger technology with privacy-preserving off-chain storage to address critical deficiencies in current healthcare IT infrastructure.

2. Problem Analysis and Threat Landscape

Contemporary healthcare data systems face structural vulnerabilities:

2.1 Architectural Vulnerabilities

- Centralized Database Architecture: Single points of failure. Compromise yields complete patient exposure.
- Weak Audit Logging: Application-layer logs can be tampered with or deleted by attackers.
- Insider Threat Vectors: Administrative accounts can access records without cryptographic proof.
- Cross-Institution Data Exchange: No technical mechanisms to verify data integrity in transit.

2.2 Real-World Attack Scenarios

- Silent Tampering: Attackers modify records and delete audit logs silently.
- Unauthorized Access: Insider threats access records outside clinical necessity.
- Credential Compromise: Stolen credentials enable access appearing legitimate.
- Ransomware with Destruction: Attackers corrupt records for extortion.

2.3 Regulatory Context

HIPAA Security Rule (45 CFR 164.312) mandates audit controls and integrity controls. MedLedger provides technical implementation through cryptographic assurance.

3. Technical Objectives and Success Criteria

3.1 Primary Objectives

1. Cryptographic Integrity Verification: Provide computational proof that records have not been modified.
2. Immutable Audit Logging: Record all access events in tamper-evident ledger.
3. Patient-Controlled Access Granularity: Patients grant/revoke access to specific records.
4. Distributed Trust Model: Eliminate single trusted administrator.

3.2 Secondary Objectives

5. Data Privacy: Sensitive medical content never written to blockchain.
6. Regulatory Compliance: Off-chain storage maintains data residency compliance.
7. Operational Performance: Authorization checks must not introduce >50ms latency.

4. Detailed System Architecture

4.1 High-Level Architecture Overview

MedLedger uses hybrid on-chain/off-chain architecture combining immutability of blockchain with privacy and performance of healthcare data systems.

```
On-Chain: Metadata, Hashes, Permissions, Audit Logs
├─ Distributed Ledger Nodes (3-7 validators)
├─ PBFT Consensus (Byzantine Fault Tolerant)
└─ Cryptographic Verification
```

```
Off-Chain: Encrypted Medical Records
├─ IPFS or Cloud Storage
├─ AES-256-GCM Encryption
└─ Locally Encrypted Copies
```

4.2 On-Chain Data Structures

Each record event contains:

```
MedicalRecordEvent:
  recordId (SHA256 hash)
  patientIdHash (hashed and salted)
  contentHash (SHA256)
  timestamp (UTC)
  eventType (CREATED, MODIFIED, ACCESS_GRANTED, etc.)
  issuerPublicKeyHash
  signature (ECDSA-P256)
  accessMetadata (with conditions)
  storageLocation (protocol, hash, keyId)
  previousEventHash (forensic chain)
  merkleProof (batch verification)
```

```
consensusRound (immutable sequence)
validators (multi-sig proof)
```

4.3 Off-Chain Storage Architecture

Medical records encrypted with AES-256-GCM:

```
EncryptedMedicalRecord:
  metadata (recordId, type, timestamps)
  encryption (AES-256-GCM, IV, salt)
  ciphertext (encrypted record)
  authenticationTag (128-bit)
  keyManagement (DEK, key references)
  integrityProof (hashes for verification)
```

5. Cryptographic Protocols and Algorithms

5.1 Key Agreement and Management

```
Key Generation:
  User Keypair: ECDSA P-256 or Ed25519 (256-bit)
  DEK Derivation: PBKDF2-SHA256
  Iterations: 100,000 (OWASP 2023 standard)
  Key Transport: ECIES (P-256) with AES-256-GCM
```

5.2 Signature and Authentication Protocol

```
Deterministic ECDSA Signing (RFC 6979):
  Input: SHA256(recordId || timestamp || contentHash || ...)
  Signature: ECDSA_P256(private_key, input_hash)
  Verification: O(1) operation, ~500 microseconds
```

5.3 Hash Chain and Merkle Tree Construction

```
Merkle Tree:
  Leaf Nodes: SHA256(MedicalRecordEvent)
  Parent Nodes: SHA256(left_child || right_child)
  Height: log2(N) where N = number of events
  Forensic Chain: Each event hashes previous event
```

6. Attribute-Based Access Control (ABAC)

Fine-grained cryptographic access control with patient authorization:

```
Access Policy includes:
```

Grantee public key hash
Attributes (role, department, specialization)
Permissions (read, write, audit, delegate)
Conditions (time window, geo-restriction, purpose)
Validity (issued, expiration, revocation)
Patient cryptographic signature

Verification:

1. Retrieve policy from ledger
2. Verify patient signature
3. Check grantee key
4. Evaluate all conditions
5. Verify access request signature
6. Consensus verification

Execution Time: <50ms target

7. Smart Contract Specification (Pseudocode)

```
contract MedicalRecordManager {

    function uploadRecord(
        patientIdHash, contentHash, storageLocation,
        signature) returns recordId {
        // Verify caller authorization
        require(verifySignature(msg.sender, signature));

        // Create and store event
        recordId = sha256(patientIdHash || now || msg.sender);
        recordIdIndex[recordId] = createEvent(
            RECORD_CREATED, contentHash, storageLocation);

        // Submit to consensus
        submitForConsensus(event);
        return recordId;
    }

    function grantAccess(recordId, granteeKeyHash, policy,
        patientSignature) {
        // Verify patient authorization
        require(verifySignature(
            getPatientPublicKey(recordId), patientSignature,
            sha256(recordId || granteeKeyHash || policy)));

        // Record grant event
        accessEvent = createEvent(ACCESS_GRANTED, ...);
        submitForConsensus(accessEvent);
    }

    function verifyIntegrity(recordId, claimedHash)
        returns bool {
        storedHash = recordIdIndex[recordId].contentHash;
```

```

    return storedHash == claimedHash;
}

function getAuditLog(recordId, startTime, endTime)
    returns AuditEvent[] {
    return queryAuditEvents(recordId, startTime, endTime);
}
}

```

8. Consensus Mechanism (Practical Byzantine Fault Tolerance)

PBFT tolerates up to $(N-1)/3$ faulty validators:

PBFTConsensusRound:

Phase 1 - PRE-PREPARE:

- Leader collects pending events, creates block
- Computes Merkle root, signs block
- Broadcasts to all followers

Phase 2 - PREPARE:

- Validators verify signatures and event validity
- Sign and broadcast sig_validator
- Collect $2f+1$ signatures (f = faulty nodes)

Phase 3 - COMMIT:

- Once $2f+1$ PREPARE signatures received
- Broadcast COMMIT message
- Block is COMMITTED (immutable) after $2f+1$ commits

Phase 4 - VIEW CHANGE:

- If leader fails, next validator becomes leader
- Process repeats

Security:

- Requires $N \geq 3f+1$ validators
- 3 validators tolerates 1 faulty
- 7 validators tolerates 2 faulty

Latency: 1-2 seconds per block

Throughput: 100-500 transactions/minute

9. Comprehensive Threat Model and Mitigations

9.1 In-Scope Threats

Database Compromise (Confidentiality): AES-256-GCM encryption. Compromise does not expose plaintext.

Record Modification Attack: Cryptographic hash verification detects within 100ms.

Unauthorized Access: Requires patient signature + grantee key + policy compliance.

Insider Threat: Immutable audit logging. Admin compromise cannot tamper logs.

Credential Compromise: Multi-signature consensus. Single credential insufficient.

Replay Attacks: Timestamp + nonce binding prevents replay.

Network Manipulation: Requires control of $2f+1$ validators simultaneously.

9.2 Out-of-Scope Threats

- Nation-state quantum computing attacks
- Physical side-channel attacks on hardware
- Compromises of >66% validators simultaneously
- Patient coercion to grant access

10. Implementation Architecture and Technology Stack

10.1 Technology Components

API Server: FastAPI (Python) with Uvicorn. <50ms p95 latency.

Cryptography: cryptography.io (OpenSSL-backed). ECDSA, AES-GCM, PBKDF2.

Ledger Engine: Custom PBFT in Python. Async event-driven.

Storage: IPFS for distributed + local encrypted database.

Key Management: HSM (Hardware Security Module) integration.

Database: PostgreSQL for metadata with WAL consistency.

Frontend: React.js with ethers.js integration.

10.2 API Specification Examples

```
POST /api/records/upload
{
  "patientIdHash": "0x3a4...",
  "contentHash": "0x7f2...",
  "storageLocation": {"protocol": "ipfs", "cid": "QmXxxx..."},
  "encryptionKeyId": "key-patient-001",
  "providerSignature": "0x308..."
}
Response: {"recordId": "0xabc...", "status": "PENDING"}

GET /api/records/{recordId}/integrity
```

```
Response: {
  "recordId": "0xabc...",
  "isValid": true,
  "lastVerified": 1707923500
}

GET /api/records/{recordId}/audit?startTime=X&endTime=Y
Response: {
  "recordId": "0xabc...",
  "events": [
    {"timestamp": 1707923400, "eventType": "RECORD_CREATED"},
    {"timestamp": 1707923456, "eventType": "ACCESS_GRANTED"}
  ]
}
```

11. Performance Characteristics and Scalability

Operation Latencies:

Hash Computation (1MB): 15ms
Signature Generation (ECDSA-P256): 8ms
Signature Verification: 12ms
AES-256-GCM Encryption (1MB): 25ms
PBFT Consensus (10 events): 1200ms
Access Policy Evaluation: 4ms
Audit Query (1M events): 4.2 seconds

System Throughput:

Read Operations: 10,000 req/sec
Metadata Queries: 5,000 req/sec
Consensus Operations: 100-500 req/sec
Records Committed: 100-500 per minute

Scalability:

Horizontal: Partition across validator groups
Vertical: Add validators for throughput
Sharding: Multiple ledgers for organizations
Archive: Historical to IPFS

Cost per transaction: <\$0.001

12. Deployment Architecture and Operations

12.1 Deployment Models

- Central Authority: Single organization operates validators.
- Peer-to-Peer: Each institution operates validators.
- Hybrid Federated: Regional clusters with federation.

12.2 Validator Node Specification

Hardware Requirements:

CPU: 8-core @ 2.5GHz (Intel Xeon)
RAM: 16GB DDR4
Storage: 500GB SSD (NVMe preferred)
Network: 1Gbps, <20ms latency to peers

Network:

3-7 validators recommended
Geographic distribution
Private VPN between validators
DDoS mitigation

Security:

Private key in HSM or encrypted vault
TLS 1.3 for communications
Linux with CIS Hardening
24/7 automated monitoring
Annual penetration testing

13. Future Roadmap and Extensions

Zero-Knowledge Proofs: Privacy-preserving access verification and audit queries.

Post-Quantum Cryptography: Lattice-based algorithms (Kyber, Dilithium).

Federated Learning: ML training on records without data exposure.

Emergency Access Protocols: Threshold crypto for life-saving access override.

HL7 FHIR Interoperability: Support standard healthcare data exchange formats.

AI Anomaly Detection: Behavioral analysis flags suspicious access patterns.

14. Conclusion

MedLedger demonstrates practical application of distributed ledger technology to healthcare data integrity. By combining cryptographic assurance with Byzantine consensus, the system provides quantifiable improvements in data integrity verification, audit trail immutability, and patient autonomy.

The hybrid on-chain/off-chain design balances transparency and immutability with privacy and performance requirements. Implementation requires coordination across multiple healthcare institutions, regulatory bodies, and technology partners.

Phased pilot deployment with 3-5 institutions and 5-7 validators is recommended as proof-of-concept before full deployment.